

⇒ 0/1 Knapsack problem by Dynamic Programming
Bottom Up approach.

⇒ Algorithm $\text{Knapsack}(W, \text{wt}[], \text{pr}[], n)$

// purpose : to maximise profit

// input : $\text{wt}[], \text{pr}[], n, W$

// output : $\text{dp}[n][W] \rightarrow$ maximum profit

create a $\text{dp}[n+1][W+1]$

for $i \leftarrow 0$ to n

for $w \leftarrow 0$ to W

if $(i == 0 \text{ or } w == 0)$

$\text{dp}[i][w] = 0$

{ else if $(\text{wt}[i-1] \leq w)$ }

$\text{dp}[i][w] = \max(\text{pr}[i-1] + \text{dp}[i-1][w - \text{wt}[i-1]],$
 $\text{dp}[i-1][w])$

{ else }

$\text{dp}[i][w] = \text{dp}[i-1][w]$

}

}

}

return $\text{dp}[n][W]$

Code:

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int knapsack (int W, int wt[], int p[], int n) {  
    int dp[n+1][W+1];  
    for (int i = 0; i <= n; i++) {  
        for (int w = 0; w <= W; w++) {  
            if (i == 0 || w == 0) {  
                dp[i][w] = 0;  
            }  
            else if (wt[i-1] <= w) {  
                dp[i][w] = max (val[i-1] + dp[i-1][w - wt[i-1]],  
                                dp[i-1][w]);  
            }  
            else {  
                dp[i][w] = dp[i-1][w];  
            }  
        }  
    }  
    return dp[n][W];  
}
```



```

int main () {
    int n, W;
    printf ("Enter no. of items: ");
    scanf ("%d", &n);
    int pr[n], wt[n];

    printf ("Enter profits of items: \n");
    for (int i=0; i<n; i++) {
        scanf ("%d", &pr[i]);
    }
    printf ("Enter weights of items: \n");
    scanf ("%d", &wt[i]);
    }
    printf ("Enter knapsack capacity: ");
    scanf ("%d", &W);

    int maxVal = knapsack (W, wt, pr, n);

    printf ("Max value in knapsack = %d \n", maxVal);

    return 0;
}

```

___/___/42

a/p: Enter no of items: 4

Enter profits of items: 12 10 20 15

Enter weights of items: 2 1 3 2

Enter knapsack capacity = 5

Max value in knapsack: 37